



Développement WEB: JavaScript Avancée

DUT Génie Informatique

Pr. Tarek AIT BAHA

t.aitbaha@uiz.ac.ma

Année universitaire : 2025/2026

Objectifs du cours

- Comprendre les bases de la programmation JavaScript,
- Variables, types de données, opérateurs,
- Structures de contrôle (boucles, conditions),
- Fonctions et portée des variables,
- Utiliser les outils de développement JavaScript,
- Manipuler le DOM et réagir aux événements,
- Utiliser des fonctionnalités modernes (ES6+),
- Comprendre la programmation asynchrone,

Objectifs d'apprentissage

À la fin de ce chapitre, vous serez capable de:

- Expliquer le rôle de JavaScript dans le développement web,
- Configurer un environnement de développement JavaScript,
- Distinguer l'exécution JS dans le navigateur vs Node.js,
- Manipuler le DOM et le BOM de façon basique.

Plan du cours

- Semaine 1 – Concepts de base et applications
- Semaine 2 – Variables et Types de données
- Semaine 3 – Fonctions
- Semaine 4 – Gestion des Événements
- **Semaine 5 – Programmation Asynchrone**
- Semaine 6 – Requêtes HTTP en JavaScript

Semaine 5:

JavaScript Synchrone vs Asynchrone

Code Synchronique - Les Bases

JavaScript est fondamentalement synchrone :

- Exécute le code une ligne après l'autre,
- Dans l'ordre exact où il est écrit.

```
1 console.log("Première ligne");  
2 console.log("Deuxième ligne");  
3 console.log("Troisième ligne");
```

Résultat:

```
Première ligne  
Deuxième ligne  
Troisième ligne
```

Code asynchrone

Parfois, nous voulons exécuter des tâches sans bloquer l'exécution du reste du code.

```
1 console.log("Étape 1");  
2  
3 setTimeout(() => {  
4     console.log("Étape 2 (retardée)");  
5 }, 2000); // Attend 2 secondes  
6  
7 console.log("Étape 3");
```

```
Étape 1  
Étape 3  
Étape 2 (retardée)
```

setTimeout est une fonction asynchrone, donc l'exécution continue pendant qu'elle attend.

Note

Si nous devons exécuter des tâches dans un ordre spécifique, nous devons gérer le **comportement asynchrone** en utilisant des techniques comme les **callbacks**, les **Promises**, ou **async/await**.

Code Synchronone

Le code synchrone est bloquant:

- Lorsqu'une fonction est appelée dans un code synchrone, l'exécution du code est bloquée jusqu'à ce que la fonction se termine.
- Cela peut poser problème lorsqu'on traite des opérations longues comme la **récupération de données depuis une API**, la **lecture d'un fichier** ou une **requête à une base de données**.
- Si l'opération prend beaucoup de temps, elle peut rendre l'application non réactive et lente.

Capacités asynchrones en JavaScript

- JavaScript est fondamentalement un langage synchrone et monothread.
- Cependant, la conception de JavaScript lui permet de gérer des opérations asynchrones grâce à des mécanismes comme les **callbacks**, les **Promises** et la syntaxe **async/await**.
- Ces fonctionnalités permettent à JavaScript d'effectuer des tâches comme des **appels API** ou des **opérations de fichier sans bloquer le thread principal, grâce à l'évent loop et à la task queue**.

Exemples de tâches asynchrones

En JavaScript (dans le navigateur ou en Node.js), la plupart des opérations **lentes** (accès réseau, lecture de fichier, temporisations...) sont exécutées de façon **asynchrone**.

- Appels API (fetch, XMLHttpRequest),
- Lecture de fichiers (File API, fs module en [Node.js](#)),
- Opérations de base de données (MongoDB, MySQL),
- Événements utilisateur (click, keypress),
- Timers (setTimeout, setInterval).

Programmation

Asynchrone vs Synchrone

Callbacks et Programmation Asynchrone

- En JavaScript (dans le navigateur ou en Node.js), la plupart des opérations lentes (accès réseau, lecture de fichier, temporisations...) sont exécutées de façon asynchrone.
- L'une des premières approches pour gérer les tâches asynchrones consistait à passer des fonctions (**callbacks**) qui s'exécutent une fois qu'une opération (comme un appel API ou la lecture d'un fichier) est terminée.
- Les **callbacks** permettent de gérer le résultat d'une opération asynchrone une fois qu'elle est terminée.

Exemple Basique setTimeout

Le plus classique : un `setTimeout` attend X millisecondes avant d'exécuter une **fonction callback**.

```
1 console.log("Début");
2
3 function afficher() {
4
5     setTimeout(() => {
6         console.log("Appel asynchrone après 2 secondes");
7     }, 2000);
8 }
9
10 afficher();
11
12 console.log("Fin");
```

- **Ordre d'exécution** : "Début" → "Fin" → (2s plus tard) "Appel asynchrone..."
- La fonction passée à `setTimeout` est un callback qui sera appelée plus tard.
- Le code continue à s'exécuter pendant que le `setTimeout` attend.

Exemple Basique `setTimeout` avec callback

Si nous voulons que “Fin” s’affiche après l’appel asynchrone, nous devons appeler `console.log("Fin")` à l’intérieur du callback de `setTimeout`.

```
console.log("Début");

// Fonction qui reçoit un callback
function afficher(callback) {
    setTimeout(() => {
        console.log("Appel asynchrone après 2 secondes");

        // exécution du callback
        callback();

    }, 2000);
}

// On passe une fonction callback
afficher(() => {
    console.log("Fin");
});
```

Exemple Basique `setTimeout` avec callback

Dans le dernier exemple :

- Nous avons défini une fonction `afficher` qui prend un callback en paramètre.
- Le `console.log("Fin")` est appelé à l'intérieur du callback passé à `afficher`.
- Cela garantit que "Fin" s'affiche après l'appel asynchrone.
- En effet, si nous avons plusieurs opérations asynchrones à enchaîner, nous serons contraints de passer des callbacks à chaque fois, ce qui peut rendre le code difficile à lire et à maintenir.
- C'est ce qu'on appelle le **callback hell** (l'enfer des callbacks).

Exemple Callback Hell

Étape 1 : Récupérer un utilisateur (getUser)

Dans cet exemple, `getUser` simule la récupération d'un utilisateur et appelle le callback avec l'utilisateur récupéré.

```
1 function getUser(callback) {
2   // Simulation d'un appel asynchrone (ex: requête réseau)
3   setTimeout(() => {
4     const user = { id: 1, name: "Alice" };
5     callback(user); // On transmet l'utilisateur récupéré via le callback
6   }, 1000);
7 }
8
9 // Exemple d'utilisation
10 getUser((user) => {
11   console.log("Étape 1 - User :", user);
12 });
```

- On définit `getUser` qui prend un **callback** en paramètre.
- Quand la fonction est exécutée, elle simule une opération asynchrone (comme une requête réseau) avec `setTimeout`.
- Après un certain temps (ici 1 seconde), elle appelle le callback pour **renvoyer** l'utilisateur.

Exemple Callback Hell

Étape 2 : Récupérer les posts d'un utilisateur (getPostsByUser)

```
1 function getPostsByUser(userId, callback) {
2   setTimeout(() => {
3     const posts = [
4       { postId: 101, title: "Mon premier post", userId },
5       { postId: 102, title: "Mon deuxième post", userId }
6     ];
7     callback(posts);
8   }, 1000);
9 }
10
11 // Exemple d'utilisation
12 getPostsByUser(1, (posts) => {
13   console.log("Étape 2 - Posts :", posts);
14 });
```

- On définit `getPostsByUser` qui reçoit un identifiant (`userId`) et un callback.
- Après avoir simulé une petite attente, elle renvoie un tableau de posts.
- Ce callback est appelé une fois que les posts sont récupérés.

Exemple Callback Hell

Étape 3 : Récupérer les commentaires d'un post (getCommentsByPost)

```
1 function getCommentsByPost(postId, callback) {
2   setTimeout(() => {
3     const comments = [
4       { commentId: 1001, text: "Super post !", postId },
5       { commentId: 1002, text: "Merci pour l'info.", postId }
6     ];
7     callback(comments);
8   }, 1000);
9 }
10
11 // Exemple d'utilisation
12 getCommentsByPost(101, (comments) => {
13   console.log("Étape 3 - Commentaires :", comments);
14 });
```

- On définit **getCommentsByPost** qui reçoit un identifiant de post (**postId**) et un **callback**.
- Après avoir simulé une petite attente, elle renvoie un tableau de commentaires associés à ce post.
- Ce callback est appelé une fois que les commentaires sont récupérés.

Exemple Callback Hell

Étape 4 : Enchaîner les callbacks (la “pyramide”)

Lorsqu'on veut enchaîner plusieurs actions asynchrones (récupérer l'utilisateur, puis ses posts, puis les commentaires du premier post), on finit par imbriquer les callbacks :

```
1  getUser((user) => {  
2    console.log("Étape 1 - User :", user);  
3  
4    getPostsByUser(user.id, (posts) => {  
5      console.log("Posts :", posts);  
6  
7      getCommentsByPost(posts[0].postId, (comments) => {  
8        console.log("Commentaires du premier post :", comments);  
9        // Si on enchaîne encore d'autres opérations asynchrones,  
10       // on continue à "plonger" encore plus loin...  
11     });  
  });
```

Callback Hell

- À chaque fois qu'on termine une opération, on appelle le **callback** suivant.
- Le problème : pour chaque opération supplémentaire, on insère un nouveau callback **à l'intérieur** du précédent, ce qui crée un code de plus en plus difficile à **lire** et à **maintenir**.
- C'est ce qu'on appelle le **callback hell** ("l'enfer des callbacks").
- **Problèmes:**
 - Code difficile à lire (indentation excessive),
 - Gestion des erreurs répétitive,
 - Difficile à déboguer,
 - Difficile à comprendre.

Solutions au Callback Hell : Les Promises

- Une **Promise** est un objet qui représente une valeur qui pourrait être disponible maintenant, dans le futur, ou jamais.
- États d'une Promise :
 - **Pending** : État initial, ni résolue ni rejetée
 - **Fulfilled** : L'opération a réussi, la promise est résolue avec une valeur
 - **Rejected** : L'opération a échoué, la promise est rejetée avec une raison

Syntaxe de base d'une Promise

```
1 // Créer une Promise
2 const maPromesse = new Promise((resolve, reject) => {
3   // Code asynchrone ici
4   if (toutVaBien) {
5     resolve("Opération réussie!"); // Résoudre avec une valeur
6   } else {
7     reject(new Error("Quelque chose a mal tourné")); // Rejeter avec erreur
8   }
9 });
```

- **resolve(value)** : Appelé lorsque l'opération réussit, transmettant une valeur.
- **reject(error)** : Appelé lorsque l'opération échoue, transmettant une erreur.

Convertir Exemple basique setTimeout en Promise

L'idée est de retourner une **promise** au lieu d'**accepter un callback**.

```
1 console.log("Début");
2
3 function afficher() {
4   return new Promise((resolve, reject) => {
5     setTimeout(() => {
6       console.log("Appel asynchrone après 2 secondes");
7       // Une fois l'opération terminée, on appelle `resolve()` au lieu de callback
8       resolve();
9     }, 2000);
10  });
11 }
12 const promise = afficher(); // Promise is now "pending"
13 console.log("Promise after creation:", promise);
```

Convertir Exemple basique setTimeout en Promise

L'idée est de retourner une **promise** au lieu d'**accepter un callback**.

```
1 // On utilise then() pour gérer le "résultat" de la promesse
2 promise.then(() => {
3   console.log("Fin");
4   // La promesse est maintenant "fulfilled"
5   console.log("Promise after resolution:", promise);
6 }).catch(error => {
7   console.error("Erreur:", error);
8 }).finally(() => {
9   console.log("Exécution terminée.");
10
11 });
```


Comment ça marche ?

1. **Promise** : Au lieu de prendre un callback en paramètre, la fonction `afficher` renvoie directement un objet Promise.
2. **resolve / reject** :
 - `resolve()` signale que tout s'est bien passé.
 - `reject()` signale une erreur.
3. `.then()` : Quand la promesse est "résolue" (après 2 secondes), le code à l'intérieur de `.then()` s'exécute.
4. `.catch()` : Si une erreur se produit, on peut utiliser `.catch()` pour gérer l'erreur.
5. `.finally()` : On peut aussi utiliser `.finally()` pour exécuter du code à la fin, qu'il y ait une erreur ou non.

Avantages des Promises

Exemple d'enchaînement de Promises

```
1  afficher()  
2    .then(() => {  
3      console.log("Je peux enchaîner une autre action...");  
4      // Retourner une autre promesse ici si besoin  
5    })  
6    .then(() => {  
7      console.log("...et continuer.");  
8    })  
9    .catch((err) => {  
10     console.error("Une erreur s'est produite :", err);  
11   })  
12   .finally(() => {  
13     console.log("Fin (qu'il y ait erreur ou non).");  
14   });
```

Avec les promises, le code s'aligne verticalement plutôt que de s'imbriquer, ce qui améliore la lisibilité et la maintenance du code.

Convertir des Fonctions Asynchrones en Promises

```
1 function getUser() {
2   return new Promise((resolve, reject) => {
3     setTimeout(() => {
4       // Simulons une probabilité d'erreur pour démontrer reject()
5       const success = Math.random() > 0.2; // 80% de chance de réussite
6
7       if (success) {
8         console.log("getUser terminé avec succès");
9         resolve({ id: 1, name: "Alice" });
10      } else {
11        console.log("getUser échoué");
12        reject(new Error("Impossible de récupérer l'utilisateur"));
13      }
14    }, 1000);
15  });
16 }
```

Convertir des Fonctions Asynchrones en Promises

```
2 function getPostsByUser(userId) {  
3   return new Promise((resolve, reject) => {  
4     // Validation du paramètre  
5     if (!userId) {  
6       return reject(new Error("userId est requis"));  
7     }  
8  
9     setTimeout(() => {  
10      console.log(`getPostsByUser terminé pour l'utilisateur ${userId}`);  
11      resolve([  
12        { postId: 101, title: "Mon premier post", userId },  
13        { postId: 102, title: "Mon deuxième post", userId }  
14      ]);  
15    }, 1000);  
16  });  
17 }
```

Convertir des Fonctions Asynchrones en Promises

```
2 function getCommentsByPost(postId) {
3   return new Promise((resolve, reject) => {
4     // Validation du paramètre
5     if (!postId) {
6       return reject(new Error("postId est requis"));
7     }
8
9     setTimeout(() => {
10      console.log(`getCommentsByPost terminé pour le post ${postId}`);
11      resolve([
12        { commentId: 1001, text: "Super post !", postId },
13        { commentId: 1002, text: "Merci pour l'info.", postId }
14      ]);
15    }, 1000);
16  });
17 }
```

Convertir des Fonctions Asynchrones en Promises

Enchaîner les Promises

On utilise la syntaxe `.then()` pour “chaîner” les appels :

```
// 1) Récupérer l'utilisateur
getUser()
  .then((user) => {
    console.log("User :", user);

    // 2) Récupérer les posts de cet utilisateur
    return getPostsByUser(user.id);
  })
  .then((posts) => {
    console.log("Posts :", posts);

    // 3) Récupérer les commentaires du premier post
    return getCommentsByPost(posts[0].postId);
  })
  .then((comments) => {
    console.log("Commentaires du premier post :", comments);
  })
  .catch((error) => {
    // Gère toutes les erreurs de la chaîne
    console.error("Erreur :", error);
  });
```

1. `getUser()` renvoie une promesse ; quand elle est résolue, on obtient `user`.
2. Dans le `.then((user) => { ... })`, on retourne `getPostsByUser(user.id)`, qui est elle-même une nouvelle promesse.
3. Le deuxième `.then((posts) => { ... })` reçoit le résultat (un tableau de posts) et **retourne** `getCommentsByPost(posts[0].postId)`.
4. Le troisième `.then((comments) => { ... })` reçoit le résultat final : les commentaires du premier post.
5. `.catch(...)` gère toute erreur potentielle sur la chaîne d'appels.

Avantages par rapport aux callbacks

1. **Code plus lisible** : Structure aplatie au lieu de imbriquée
2. **Meilleure gestion des erreurs** : Un seul `.catch()` peut gérer les erreurs de toute la chaîne
3. **Composition** : Facile à composer avec d'autres Promises
4. **Gestion propre du flux asynchrone** : `.then()` ne s'exécute que lorsque la Promise est résolue

Async/Await : Une syntaxe plus simple pour les Promises

- Syntaxe moderne pour travailler avec des Promises,
- Rend le code asynchrone plus lisible,
- Utilise **try/catch** pour la gestion des erreurs.

Async/Await : Une syntaxe plus simple pour les Promises

- `async` : marque une fonction comme asynchrone, permettant d'utiliser `await` à l'intérieur.
- `await` : suspend l'exécution de la fonction jusqu'à ce que la Promise soit résolue ou rejetée.
- `try/catch` : gère les erreurs de manière similaire à la gestion des exceptions synchrones.

Async/Await : Une syntaxe plus simple pour les Promises

- La création de Promises ne change pas, mais la façon de les utiliser devient plus simple et plus lisible.
- Au lieu de chaîner les `.then()`, on utilise `await` pour attendre la résolution d'une Promise.
- Cela permet d'écrire du code asynchrone de manière plus linéaire, comme du code synchrone.

Exemple basique de `async/await`

```
1 // Exemple avec Promises
2 function afficher() {
3     return new Promise((resolve) => {
4         setTimeout(() => {
5             console.log("Appel asynchrone après 2 secondes");
6             resolve("Résultat après le setTimeout");
7         }, 2000);
8     });
9 }
10
11 afficher().then((result) => {
12     console.log("Fin - Resultat :", result);
13 });
```

Exemple basique de **async/await**

```
1 // On déclare une fonction asynchrone
2 async function demoAsyncAwait() {
3   // On attend que la Promise soit résolue
4   const result = await afficher();
5   console.log("Fin - Résultat :", result);
6 }
7
8 // On appelle la fonction asynchrone
9 demoAsyncAwait();
```

Exemple basique de **async/await**

```
1 // Exemple avec Promises
2 function afficher() {
3   return new Promise((resolve) => {
4     setTimeout(() => {
5       console.log("Appel asynchrone après 2 secondes");
6       resolve("Résultat après le setTimeout");
7     }, 2000);
8   });
9 }
10
11 afficher().then((result) => {
12   console.log("Fin - Resultat :", result);
13 });
```

```
1 // On déclare une fonction asynchrone
2 async function demoAsyncAwait() {
3   // On attend que la Promise soit résolue
4   const result = await afficher();
5   console.log("Fin - Résultat :", result);
6 }
7
8 // On appelle la fonction asynchrone
9 demoAsyncAwait();
```

- **async function demoAsyncAwait()** : on marque la fonction comme asynchrone.
- **await afficher()** : on suspend l'exécution de la fonction jusqu'à ce que la promesse renvoyée par **afficher()** soit terminée.
 - En retour, on obtient la valeur transmise par **resolve()**, ici **Résultat après le setTimeout**.
- Avantage : on écrit le code **comme s'il était synchrone**, sans **.then()**, ce qui le rend plus **lisible**.

Exemple user, post, comment avec async/await

Partons du même principe, mais appliquons **async/await** aux fonctions **getUser**, **getPostsByUser** et **getCommentsByPost**.

```
// Chaîne de Promises
getUser()
  .then((user) => {
    console.log("User :", user);
    return getPostsByUser(user.id);
  })
  .then((posts) => {
    console.log("Posts :", posts);
    return getCommentsByPost(posts[0].postId);
  })
  .then((comments) => {
    console.log("Commentaires du premier post :", comments);
  })
  .catch((error) => {
    console.error("Erreur :", error);
  });
```

```
function getUser() {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("getUser terminé");
      resolve({ id: 1, name: "Alice" });
    }, 1000);
  });
}

function getPostsByUser(userId) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("getPostsByUser terminé");
      resolve([
        { postId: 101, title: "Mon premier post", userId },
        { postId: 102, title: "Mon deuxième post", userId }
      ]);
    }, 1000);
  });
}

function getCommentsByPost(postId) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("getCommentsByPost terminé");
      resolve([
        { commentId: 1, text: "Super article !" },
        { commentId: 2, text: "Merci pour ce post !" }
      ]);
    }, 1000);
  });
}
```

Exemple user, post, comment avec async/await

```
async function fetchUserPostsAndComments() {
  try {

    // 1) Récupérer l'utilisateur
    const user = await getUser();
    console.log("User :", user);

    // 2) Récupérer les posts de l'utilisateur
    const posts = await getPostsByUser(user.id);
    console.log("Posts :", posts);

    // 3) Récupérer les commentaires du premier post
    const comments = await getCommentsByPost(posts[0].postId);
    console.log("Commentaires du premier post :", comments);

  } catch (error) {
    // Gérer les erreurs
    console.error("Une erreur s'est produite :", error);
  }
}

// Appel de la fonction
fetchUserPostsAndComments();
```

Avantages de Async/Await

1. **Syntaxe Simple et Lisible** : Le code ressemble presque à du code synchrone, ce qui le rend plus facile à comprendre.
2. **Gestion des Erreurs Améliorée** : Utilisez un seul bloc `try/catch` pour gérer les erreurs à n'importe quelle étape.
3. **Débogage Plus Simple** : Plus facile de suivre l'exécution et de déboguer le code.
4. **Pas d'Imbrication** : Fini la pyramide des callbacks et même les chaînes de `.then()`.